《电子系统设计实战》
个人课程报告

项目名称	ESP32 蓝牙+按键智能窗帘控制系统
小组编号	26
学生姓名	李志豪
学号	2311003
本人分工	蓝牙 APP 开发、系统调试
提交日期	2026.6.18

1. 项目概况

本项目名称为 ESP32 蓝牙+按键智能窗帘控制系统，面向传统环境监测与窗帘控制设备功能单一、布线繁琐、智能化程度低，以及宿舍、教室、实验室、小型仓储等场景对环境自动化管控的实际需求，主要实现光照强度的实时监测、智能窗帘的自动启闭以及远程无线操控等一体化功能。系统由 ESP32 主控模块、光照传感器模块、舵机与声光提醒执行机构、蓝牙通信模块、物理按键与远程交互界面和板载稳压电源模块等组成。

小组成员分工如下表所示。

成员	主要分工	负责内容说明
刘博帅	PCB 电路板设计、嵌入式软件	绘制电路板、嵌入式程序开发、焊接电路
李佳杰	PCB 原理图设计、窗帘机械结构	芯片选型、原理图设计、机械结构设计搭建
李志豪	蓝牙 APP 开发、系统调试	手机端 APP 开发、蓝牙通信设计、软硬件调试
李志豪	蓝牙 APP 开发、系统调试	手机端 APP 开发、蓝牙通信设计、软硬件调试

2. 本人承担的任务与目标

本人在项目中主要负责手机端 APP 开发、蓝牙通信设计与软硬件调试，该模块在系统中的作用是构建移动端人机交互界面，实现手机与 ESP32 的蓝牙数据双向传输，并通过系统联调确保窗帘控制及光照控制及及提醒功能稳定运行。

本人负责模块需要完成的具体任务如下：

任务编号	具体任务	目标或指标	完成情况
T1	手机端控制 APP 界面设计与功能开发	界面操作流畅，蓝牙按键控制延迟低	已完成
T2	手机与 ESP32 主控的蓝牙无线通信协议设计	实现手机与主控板的稳定配对，指令传输准确无丢包	已完成
T3	软硬件系统联调与功能测试	解决传感器、舵机和声光提示的软硬件冲突，确保整机稳定运行	已完成

3. 个人负责模块设计

本人负责手机端 APP 开发、蓝牙通信设计与软硬件系统联调。考虑到数据高频双向传输的稳定性需求，选用基于 SPP 协议的经典蓝牙方案，并在 Flutter 框架下开发了集环境监测与窗帘控制于一体的直观卡片式界面。APP 通过发送 OPEN CLOSE AUTO 等简易文本指令控制硬件，同时在后台实时环境数据（光照）来更新界面数值。在可靠性方面，通过帧头尾校验机制防止数据损坏闪退，并加入了断线自动重连逻辑应对异常断开。该方案具有交互反馈直观、数据传输延迟低、系统抗干扰能力强且便于软硬件闭环联调的优点。

4. 具体实现过程

1.开发环境与实现步骤

开发环境基于 Flutter 跨平台框架与 Dart 语言，在 VS Code 中编写。硬件端配合 ESP32 及光敏电阻传感器进行数据闭环。

实现步骤首先是设计单页卡片式 UI，用于显示光照强度和控制窗帘。接着在底层引入 flutter_bluetooth_serial 插件，通过设备 MAC 地址与 ESP32 建立经典蓝牙连接。最后实现双向数据交互，即 APP 端发送控制指令，同时后台监听并解析硬件上传的数据。

2. 核心代码与逻辑解释，APP 端接收蓝牙数据并实时更新 UI 的核心代码如下：

```
connection.input.listen((data) {
String rawData = utf8.decode(data);
if (rawData.startsWith("$") && rawData.endsWith("#")) {
List<String> values = rawData.substring(1, rawData.length - 1).split(",");
setState(() {
lightIntensity = values[0];
curtainStatus = values[1];
});
}
});
```

代码解析：APP 后台持续监听蓝牙字节流并解码为字符串。当识别到以“$”开头、“#”结尾的完整数据帧时，截取中间的有效内容并通过逗号进行分割。随后调用 setState 函数，实时更新光照强度与窗帘状态，触发界面 UI 的动态刷新。

3. 调试过程与关键改进

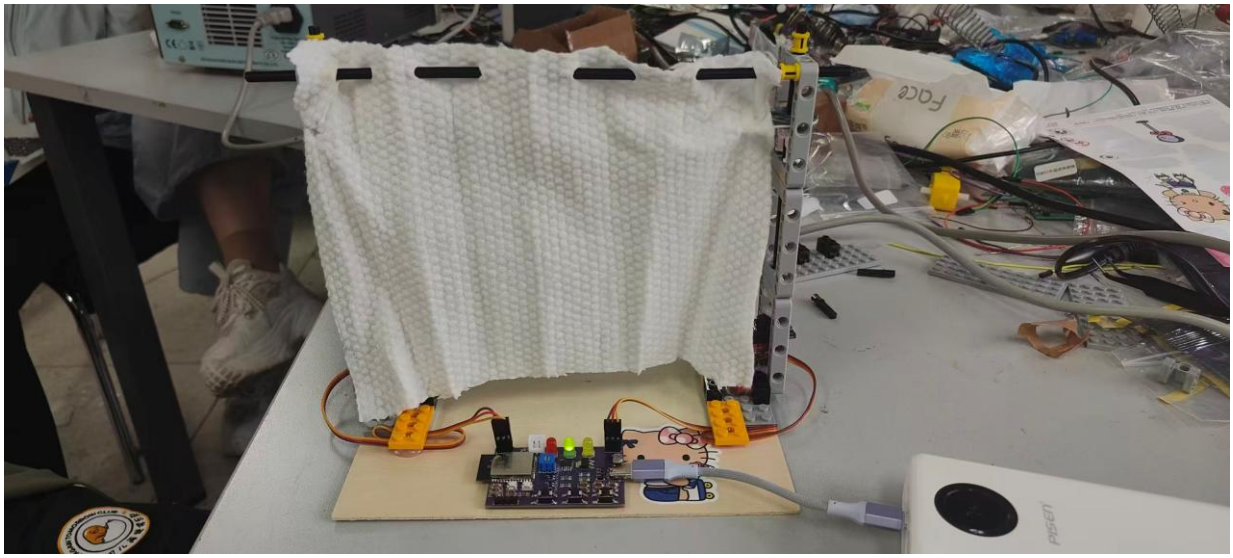
解决自动与手动模式控制冲突：在系统联调中，若系统处于自动光控模式，用户在 APP 端点击手动按钮会与硬件自动逻辑冲突，导致电机运行混乱。关键改进是在 APP 端引入了模式锁机制：当系统切换为自动模式时，APP 端的手动控制按钮（开启、关闭、停止）在代码层面自动置灰并禁用点击事件，从源头上避免了人机冲突，保障了系统运行的稳定性。

5. 调试、测试与结果分析

模块测试一：蓝牙连接稳定性与自动重连测试。测试方法为手机 APP 与 ESP32 成功建立连接后，手动断开硬件电源模拟断线，10 秒后重新上电，观察 APP 的交互状态。预期结果是 APP 在断线后能立即提示，且在重新上电后自动恢复连接。实际结果为硬件断电 2 秒内，APP 界面顶部状态栏切换为已断开，重新上电后，APP 在 3 秒内通过后台重连机制成功恢复通信。问题分析表明，断线重连逻辑触发灵敏，极大增强了系统在实际使用中的抗干扰能力。

模块测试二：数据接收解析与 UI 更新测试。测试方法为通过遮挡传感器或使用手电筒照射来改变光照，对比硬件串口输出的数据帧与手机 APP 界面的显示数值。在测试数据方面，当硬件串口发送数据帧 $80,0# 时，APP 界面同步显示光照强度为 80Lux 且窗帘关闭；当串口发送数据帧 $850,1# 时，APP 同步更新为 850Lux 且窗帘开启。预期与实际结果完全一致，数据更新延迟低于 100 毫秒，且连续高频刷新无乱码。问题分析表明，基于双标志位验证的解析逻辑运行稳定，消除了无线传输中数据分包的影响。

模块测试三：模式切换与控制按钮互锁测试。测试方法是在 APP 端将系统切换为自动模式，尝试点击窗帘的开启和关闭按钮，随后切回手动模式并点击开启。预期结果是自动模式下控制按钮置灰并禁用，手动模式下能正常控制电机。实际结果为切换至自动模式后，开启和关闭按钮立即呈现置灰状态且无法点击，从源头上杜绝了操作冲突；切回手动模式后，按钮恢复常态，点击开启后，硬件串口瞬间接收到控制指令 OPEN 且电机开始运转。问题分析表明，APP 端的状态锁机制工作正常，完全消除了人机操作与硬件自动运行之间的冲突。



演示视频链接： https://b23.tv/ld7zYDq

蓝牙 APP 代码链接：https://github.com/ZhihaoLee123/curtain_app

6. 与其他模块的接口和协同

本项目为小组系统设计项目，因此个人报告中还应说明本人模块如何与其他成员负责的模块连接和协同。

接口对象	接口内容	数据/信号格式	协同或联调情况
刘博帅（嵌入式软件）	发送控制指令并接收传感器回传数据	经典蓝牙 SPP 协议	共同制定了蓝牙数据帧格式。联调阶段通过串口抓包，成功实现 APP 下发指令与传感器数据回传的闭环。
李佳杰（原理图与结构）	调试 APP 按键与窗帘电机运行状态的匹配度	物理开合行程与 APP 端状态显示	协同完成了机械限位调试。确保 APP 端发送“开启”指令时，窗帘电机的物理运行距离与 APP 端显示的“开启”状态一致。

7. 问题记录与解决方法

问题编号	问题现象	原因分析	解决方法	解决效果
P1	蓝牙数据传输高频刷新时，APP 界面偶发乱码甚至直接闪退。	蓝牙串口字节流传输存在分包和粘包，APP 端直接截取字符导致了数组越界。	在 APP 端引入全局缓冲区，设计基于首尾标志位（“$”和“#”）的动态校验和帧拼接算法。	成功消除了数据分片导致的解析错误，高频刷新下 APP 运行稳定无闪退。
P2	自动光控模式下，用户若在 APP 端点击控制按键，电机会出现抖动或逻辑混乱。	硬件的自动运行逻辑与 APP 下发的手动控制指令产生冲突，同时在控制电机。	在 APP 端加入控制状态锁。当系统处于自动模式时，自动将 APP 端的手动按键置灰并禁用点击事件。	杜绝了人机操作冲突，只有切换至手动模式时按钮才可操作，系统控制逻辑清晰。

8. 个人贡献总结与课程收获

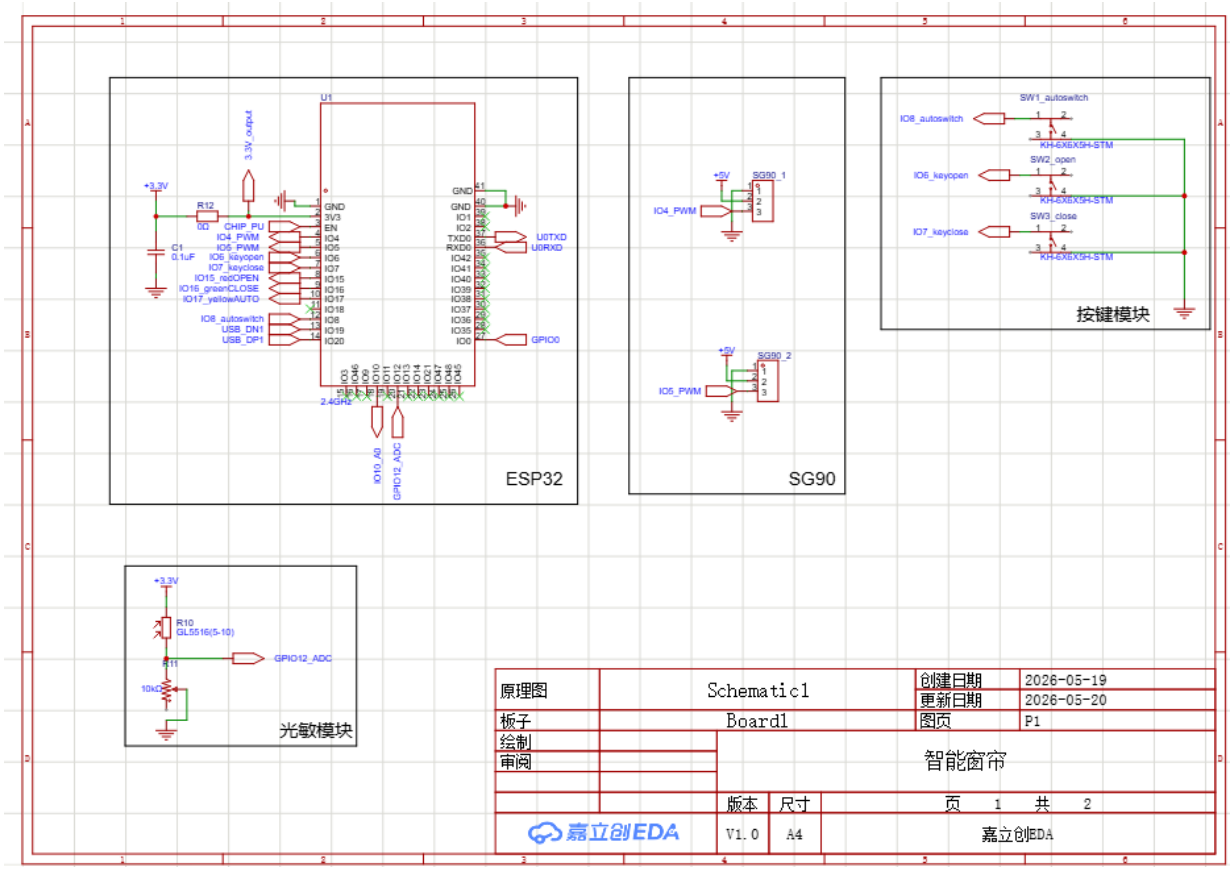
本人在本项目中完成了手机端 Flutter APP 的设计与开发、经典蓝牙通信协议的搭建、数据流拼接与解析逻辑编写，以及自动重连与控制锁机制的独立开发。并与小组成员共同完成了软硬件系统联合调试、通信数据帧格式制定以及电机控制与 APP 状态匹配调试。通过本项目，本人对蓝牙通信协议、Flutter 跨平台开发以及软硬件系统联调有了更深入的理解。

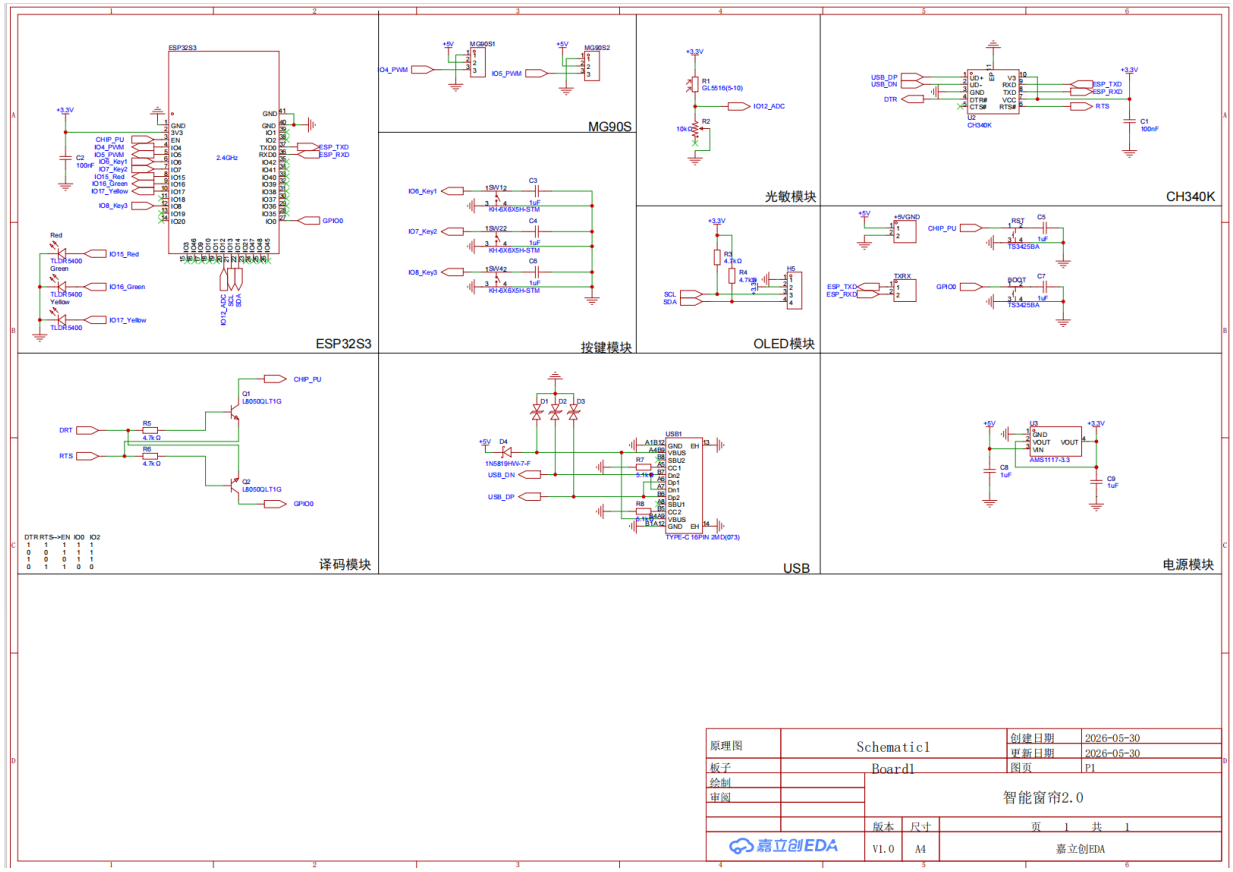
本项目仍存在蓝牙通信功耗较高（未使用 BLE 低功耗蓝牙）以及控制距离受限（仅限本地蓝牙范围，缺少 Wi-Fi 远程云控制）等问题。后续可以从低功耗设计（将经典蓝牙升级为 BLE 低功耗蓝牙）、算法优化（接入物联网云平台实现远程控窗）以及异常保护机制（增加电机过载堵转检测并在 APP 端进行报警提示）等方面继续改进。

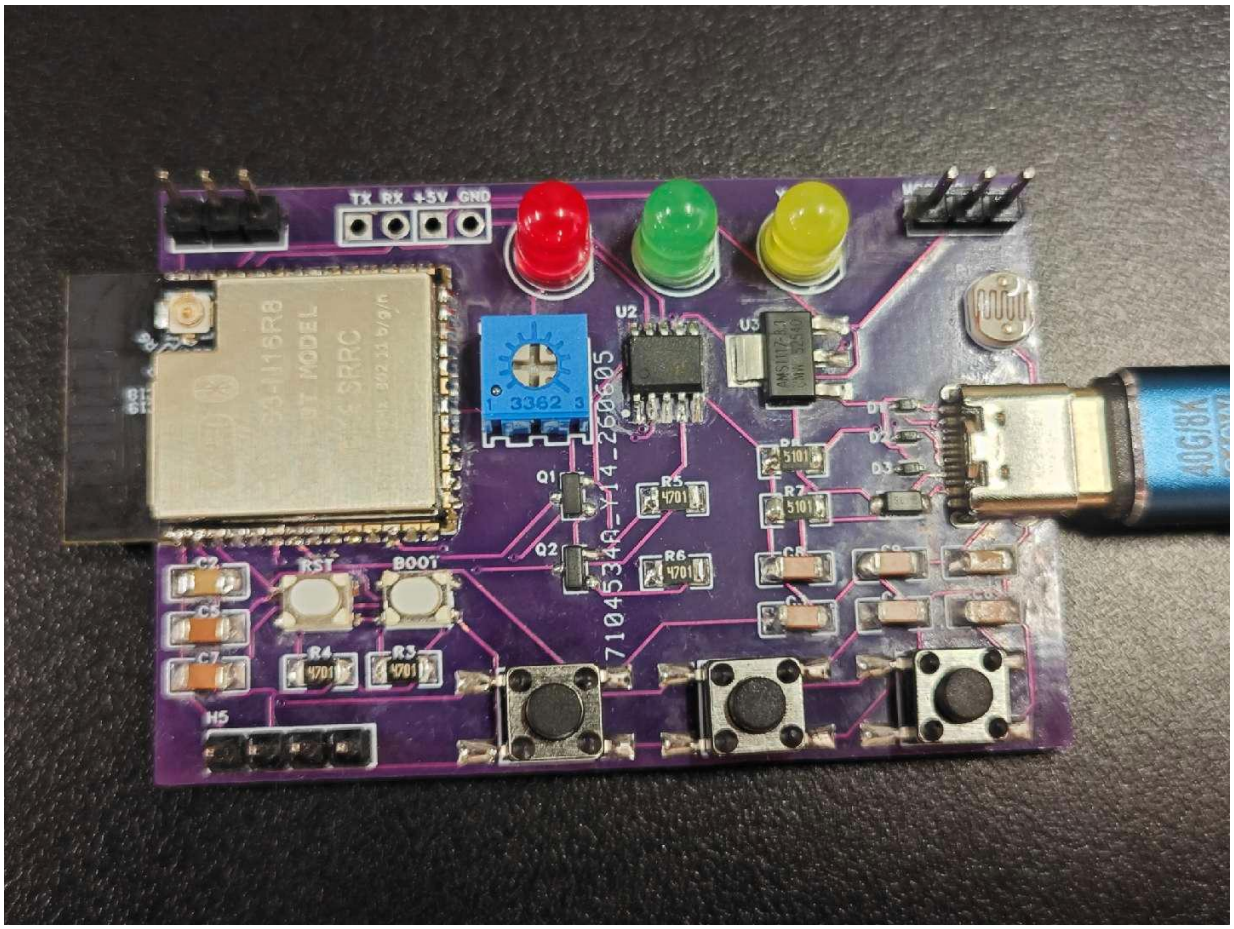
附录

演示视频链接： https://b23.tv/ld7zYDq

蓝牙 APP 代码链接：https://github.com/ZhihaoLee123/curtain_app






TX RX +5V GND
U2
U3
3362
Q1
Q2
R5
4701
R6
4701
R7
5101
C2
C5
C7
RST
BOOT
R4
4701
R3
4701
H5
7104534-14_250605